

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни: «Об'єктно-орієнтоване програмування»

на тему: «Рефакторинг процедурного коду та об'єктно-орієнтоване проєктування програмних систем»

Виконав: студент групи ІПС-22

Шевчик Роман Олександрович

1. МЕТА РОБОТИ

Ознайомитися з принципами об'єктно-орієнтованого програмування (ООП), освоїти інструменти візуального моделювання програмних систем (UML) та системи контролю версій (Git). Набути практичних навичок проведення рефакторингу існуючого процедурного коду, рознесення логіки за принципом єдиного обов'язку (SRP), а також покриття оновленої архітектури автоматизованими юніт-тестами.

2. ПОСТАНОВКА ЗАВДАННЯ

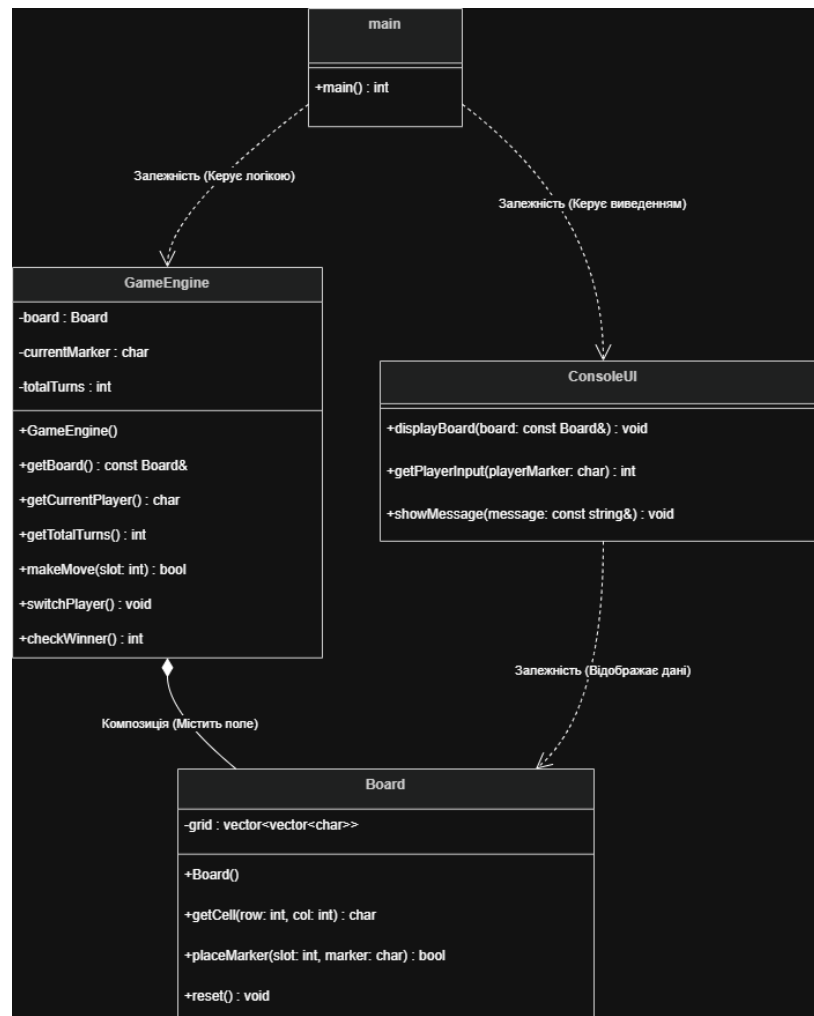
1. Розробити або взяти за основу існуючу процедурну реалізацію на мові C++.
2. Зафіксувати початковий стан коду в системі контролю версій Git та помітити його відповідним тегом.
3. Провести системний аналіз предметної області, виділити ключові сутності та розробити концептуальну UML-діаграму класів (To-Be модель).
4. Виконати рефакторинг програми, перевівши її на ООП-рейки. Розділити логіку на окремі класи (модулі): збереження стану поля, ігровий двигунець, інтерфейс користувача.
5. Написати комплекс автоматизованих юніт-тестів для перевірки коректності роботи ігрової логіки.
6. Структурувати репозиторій за стандартами розробки (папки src, tests, docs) та оформити інструкцію у файлі README.md.

3. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА UML-ПРОЄКТУВАННЯ

В ході аналізу гри «Хрестики-Нулики» було виділено три основні абстракції:

- **Ігрове поле (Board):** відповідає за збереження матриці 3x3, перевірку доступності клітинок та безпосередній запис маркерів.
- **Двигунець гри (GameEngine):** керує станом ігрового процесу, відстежує чергу ходів гравців, аналізує поле на наявність виграшних комбінацій або нічиєї.
- **Інтерфейс користувача (ConsoleUI):** відповідає за текстове відображення стану системи в консолі та зчитування дій користувача.

Зв'язок між GameEngine та Board реалізовано за допомогою **композиції**, оскільки життєвий цикл ігрового поля повністю керується двигунцем гри.



4. СТРУКТУРА ПРОЄКТУ ТА КОНТРОЛЬ ВЕРСІЙ

Для ведення розробки було налаштовано Git-репозиторій. Історія комітів відображає еволюцію проєкту від процедурного коду до фінальної структурованої ООП-архітектури.

```
Docs: Update README.md with project description and compilation guide
Docs: Update README.md with project description and compilation guide
Git: Add .gitignore to exclude compiled binary files from repository
Test: Implement automated unit tests for checking GameEngine OOP logic
Docs: Add UML Class diagram project and image for To-Be model
Code: Add original procedural code for history and comparison
Refactor: Transform procedural code into OOP design with Board, GameEngine, and ConsoleUI
(tag: v1.0-procedural) Initial commit: Base procedural Tic-Tac-Toe code before refactoring
Initial commit
```

Файлова структура фінальної версії проєкту має такий вигляд:

- src/ — вихідний ООП-код програми (Board.hpp, GameEngine.hpp, ConsoleUI.hpp, new_main.cpp).
- tests/ — автоматичні Unit-тести (tests.cpp).
- docs/ — проєктні UML-схеми.
- procedural_version/ — початкова версія для порівняння.

.gitignore — файл конфігурації для виключення тимчасового сміття компілятора.

5. ВИХІДНИЙ КОД ПРОГРАМИ (ООП ВЕРСІЯ)

Клас Board.hpp

```
Board.hpp > ...
#pragma once
#include <vector>

class Board {
private:
    std::vector<std::vector<char>> grid;

public:
    Board() {
        grid = std::vector<std::vector<char>>(3, std::vector<char>(3, ' '));
    }

    char getCell(int row, int col) const {
        return grid[row][col];
    }

    bool placeMarker(int slot, char marker) {
        int row = (slot - 1) / 3;
        int col = (slot - 1) % 3;

        if (slot < 1 || slot > 9 || grid[row][col] != ' ') {
            return false;
        }

        grid[row][col] = marker;
        return true;
    }

    void reset() {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                grid[i][j] = ' ';
            }
        }
    }
};
```

Клас GameEngine.hpp

```

GameEngine.hpp > GameEngine > getTotalTurns() const
#pragma once
#include "Board.hpp"

class GameEngine {
private:
    Board board;
    char currentMarker;
    int totalTurns;

public:
    GameEngine() : currentMarker('X'), totalTurns(0) {}

    const Board& getBoard() const { return board; }
    char getCurrentPlayer() const { return currentMarker; }
    int getTotalTurns() const { return totalTurns; }

    bool makeMove(int slot) {
        if (board.placeMarker(slot, currentMarker)) {
            totalTurns++;
            return true;
        }
        return false;
    }

    void switchPlayer() {
        currentMarker = (currentMarker == 'X') ? 'O' : 'X';
    }

    int checkWinner() const {
        for (int i = 0; i < 3; i++) {
            if (board.getCell(i, 0) == board.getCell(i, 1) && board.getCell(i, 1) == board.getCell(i, 2) && board.getCell(i, 0) != ' ')
                return 1;
            if (board.getCell(0, i) == board.getCell(1, i) && board.getCell(1, i) == board.getCell(2, i) && board.getCell(0, i) != ' ')
                return 1;
        }
        // Перевірка діагоналей
        if (board.getCell(0, 0) == board.getCell(1, 1) && board.getCell(1, 1) == board.getCell(2, 2) && board.getCell(0, 0) != ' ')
            return 1;
        if (board.getCell(0, 2) == board.getCell(1, 1) && board.getCell(1, 1) == board.getCell(2, 0) && board.getCell(0, 2) != ' ')
            return 1;

        if (totalTurns == 9) return -1;
        return 0;
    }
};

```

Клас ConsoleUI.hpp

```

/workspaces/lab1/src/ConsoleUI.hpp (preview) message(const std::string &) const
#pragma once
#include <iostream>
#include "GameEngine.hpp"

class ConsoleUI {
public:
    void displayBoard(const Board& board) const {
        std::cout << "-----\n";
        for (int i = 0; i < 3; i++) {
            std::cout << "| ";
            for (int j = 0; j < 3; j++) {
                std::cout << board.getCell(i, j) << " | ";
            }
            std::cout << "\n-----\n";
        }
    }

    int getPlayerInput(char playerMarker) const {
        int slot;
        std::cout << "Гравець " << playerMarker << ", виберіть клітинку (1-9): ";
        std::cin >> slot;
        return slot;
    }

    void showMessage(const std::string& message) const {
        std::cout << message << std::string("\n");
    }
};

```

Точка входу new_main.cpp

```

new_main.cpp> main0=ew_main.cpp (preview)
#include "GameEngine.hpp"
#include "ConsoleUI.hpp"

int main() {
    GameEngine game;
    ConsoleUI ui;

    ui.showMessage("Вітаємо у гри Хрестики-Нулики (ООП версія!");
    ui.displayBoard(game.getBoard());

    while (true) {
        int slot = ui.getPlayerInput(game.getCurrentPlayer());

        if (!game.makeMove(slot)) {
            ui.showMessage("Невірний хід! Спробуйте ще раз.");
            continue;
        }

        ui.displayBoard(game.getBoard());
        int status = game.checkWinner();

        if (status == 1) {
            ui.showMessage("Вітаємо! Гравець " + std::string(1, game.getCurrentPlayer()) + " переміг!");
            break;
        } else if (status == -1) {
            ui.showMessage("Нічия!");
            break;
        }

        game.switchPlayer();
    }

    return 0;
}

```

6. РЕЗУЛЬТАТИ ТЕСТУВАННЯ

Для автоматизованої перевірки бізнес-логіки системи було розроблено набір юніт-тестів (файл tests/tests.cpp). Перевірено такі критичні сценарії:

1. Ініціалізація початкового стану гри (порожнє поле, хід за 'X').
2. Обробка коректного ходу гравця.
3. Блокування некоректного ходу в уже зайняту клітинку.
4. Правильність автоматичного перемикавання черги ходів.
5. Фіксація фінального стану виграшу за горизонтальною лінією.

Всі розроблені тести успішно пройшли перевірку, що гарантує стабільність роботи ігрового двигунця.

```
@romashewa1456 →/workspaces/lab1 (main) $ ./run_tests
--- ЗАПУСК ЮНІТ-ТЕСТІВ (ООП ВЕРСІЯ) ---
Test InitialState: PASSED
Test ValidMove: PASSED
Test InvalidMove: PASSED
Test PlayerSwitching: PASSED
Test WinCondition Horizontal: PASSED
Всі ООП-тести успішно пройдені!
```

7. ВИСНОВКИ

Під час виконання лабораторної роботи було успішно проведено рефакторинг процедурного коду гри «Хрестики-Нулики» в об'єктно-орієнтовану структуру. Розділення коду на ізольовані класи (Board, GameEngine, ConsoleUI) дозволило досягти високого рівня модульності, гнучкості та спростило подальше супроводження програми.

Було здобуто практичний досвід роботи з UML-модельюванням у Draw.io для побудови діаграм класів, опановано базові концепції інкапсуляції та композиції в C++, а також закріплено навички роботи з автоматизованими тестами (Unit Testing) та управління індексами у системі Git. Проєкт повністю готовий до подальшого розширення архітектури.